



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Velazquez Caudillo Osbaldo

N° de Cuenta: 320341704

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 31/08/25

CALIFICACIÓN: _____

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Dibujar las iniciales de sus nombres, cada letra de un color diferente

```
GLfloat vertices_letras[] = {
    //X   Y   Z       R   G   B
    // LETRA O (Azul)
    -0.9f,-0.5f,0.0f,  0.0f,0.0f,1.0f,
    -0.4f,-0.5f,0.0f,  0.0f,0.0f,1.0f,
    -0.4f,-0.3f,0.0f,  0.0f,0.0f,1.0f,

    -0.9f,-0.5f,0.0f,  0.0f,0.0f,1.0f,
    -0.4f,-0.3f,0.0f,  0.0f,0.0f,1.0f,
    -0.9f,-0.3f,0.0f,  0.0f,0.0f,1.0f,

    -0.9f,0.3f,0.0f,   0.0f,0.0f,1.0f,
    -0.4f,0.3f,0.0f,   0.0f,0.0f,1.0f,
    -0.4f,0.5f,0.0f,   0.0f,0.0f,1.0f,

    -0.9f,0.3f,0.0f,   0.0f,0.0f,1.0f,
    -0.4f,0.5f,0.0f,   0.0f,0.0f,1.0f,
    -0.9f,0.5f,0.0f,   0.0f,0.0f,1.0f,

    -0.9f,-0.3f,0.0f,  0.0f,0.0f,1.0f,
    -0.8f,-0.3f,0.0f,  0.0f,0.0f,1.0f,
    -0.8f,0.3f,0.0f,   0.0f,0.0f,1.0f,

    -0.9f,-0.3f,0.0f,  0.0f,0.0f,1.0f,
    -0.8f,0.3f,0.0f,   0.0f,0.0f,1.0f,
    -0.9f,0.3f,0.0f,   0.0f,0.0f,1.0f,

    -0.5f,-0.3f,0.0f,  0.0f,0.0f,1.0f,
    -0.4f,-0.3f,0.0f,  0.0f,0.0f,1.0f,
    -0.4f,0.3f,0.0f,   0.0f,0.0f,1.0f,

    -0.5f,-0.3f,0.0f,  0.0f,0.0f,1.0f,
    -0.4f,0.3f,0.0f,   0.0f,0.0f,1.0f,
    -0.5f,0.3f,0.0f,   0.0f,0.0f,1.0f,

    // LETRA V (Roja)
    -0.1f,-0.5f,0.0f,  1.0f,0.0f,0.0f,
    0.1f,-0.5f,0.0f,  1.0f,0.0f,0.0f,
    -0.1f,-0.3f,0.0f,  1.0f,0.0f,0.0f,

    0.1f,-0.5f,0.0f,  1.0f,0.0f,0.0f,
    0.1f,-0.3f,0.0f,  1.0f,0.0f,0.0f,
    -0.1f,-0.3f,0.0f,  1.0f,0.0f,0.0f,

    0.0f,-0.3f,0.0f,  1.0f,0.0f,0.0f,
    -0.1f,0.5f,0.0f,  1.0f,0.0f,0.0f,
    -0.1f,-0.3f,0.0f,  1.0f,0.0f,0.0f,

    0.0f,-0.3f,0.0f,  1.0f,0.0f,0.0f,
    0.1f,-0.3f,0.0f,  1.0f,0.0f,0.0f,
    0.1f,0.5f,0.0f,   1.0f,0.0f,0.0f,

    -0.1f,-0.5f,0.0f,  1.0f,0.0f,0.0f,
    -0.1f,0.5f,0.0f,  1.0f,0.0f,0.0f,
    -0.3f,0.5f,0.0f,   1.0f,0.0f,0.0f,

    0.1f,-0.5f,0.0f,  1.0f,0.0f,0.0f,
    0.3f,0.5f,0.0f,   1.0f,0.0f,0.0f,
    0.1f,0.5f,0.0f,   1.0f,0.0f,0.0f,

    // LETRA C (Gris)
    0.4f,-0.5f,0.0f,  0.5f,0.5f,0.5f,
    0.5f,-0.5f,0.0f,  0.5f,0.5f,0.5f,
    0.4f,-0.3f,0.0f,  0.5f,0.5f,0.5f,

    0.5f,-0.5f,0.0f,  0.5f,0.5f,0.5f,
    0.5f,-0.3f,0.0f,  0.5f,0.5f,0.5f,
    0.4f,-0.3f,0.0f,  0.5f,0.5f,0.5f,

    0.4f,0.3f,0.0f,   0.5f,0.5f,0.5f,
    0.5f,0.5f,0.0f,  0.5f,0.5f,0.5f,
    0.4f,0.5f,0.0f,   0.5f,0.5f,0.5f,

    0.4f,-0.3f,0.0f,  0.5f,0.5f,0.5f,
    0.5f,-0.3f,0.0f,  0.5f,0.5f,0.5f,
    0.6f,0.3f,0.0f,   0.5f,0.5f,0.5f,

    0.4f,-0.3f,0.0f,  0.5f,0.5f,0.5f,
    0.6f,0.3f,0.0f,   0.5f,0.5f,0.5f,
    0.4f,0.3f,0.0f,   0.5f,0.5f,0.5f,

    MeshColor* letras = new MeshColor();
    letras->CreateMeshColor(vertices_letras, sizeof(vertices_letras) / sizeof(GLfloat));
    meshColorList.push_back(letras);
};
```

Aquí creamos los vértices de cada letra con su respectivo color, en mi caso la “O” azul la “V” roja y la “C” color gris.

```
MeshColor* letras = new MeshColor();
letras->CreateMeshColor(vertices_letras, sizeof(vertices_letras) / sizeof(GLfloat));
meshColorList.push_back(letras);
```

Aquí lo que hacemos es guardar cuántos vértices tiene (con el cálculo $\text{sizeof}(\text{vertices_letras})/\text{sizeof}(\text{GLfloat}) \rightarrow \text{total de floats} / 6 = \text{cantidad de vértices}$).

Selecciono el shader que usa color por vértice.

```
shaderList[5].useShader();
uniformModel = shaderList[5].getModelLocation();
uniformProjection = shaderList[5].getProjectLocation();
```

En mi caso es el 5 por que lo agregue hasta el último de mi lista.

Y posteriormente paso a dibujar la malla de letras (ya está en `meshColorList[0]`)

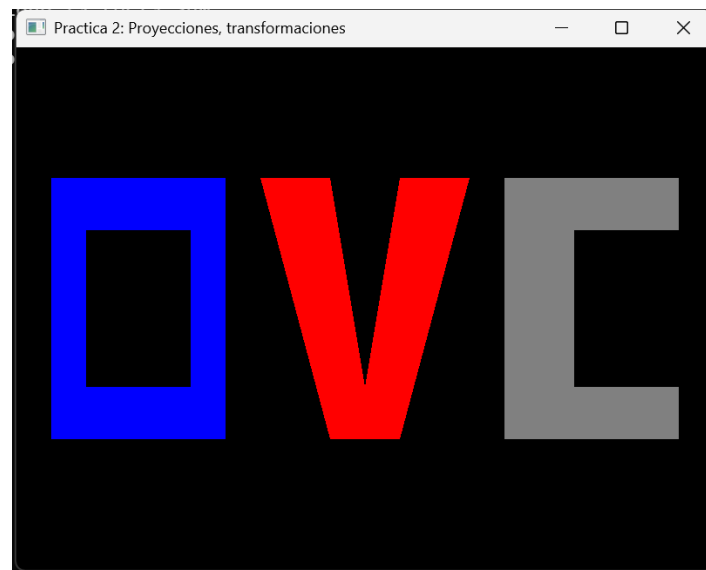
```
//Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geométricas
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
//
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA y se envían al shader
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshColorList[0] -> RenderMeshColor();
```

Para poder visualizar las letras des comentar

```
mainWindow.Initialise();
//CrearLetrasyFiguras(); //usa MeshColor, índices en MeshColorList
CreaPiramide(); //índice 0 en MeshList
```

```
/*
//Actividad Letras
//Para las letras hay que usar el segundo set de shaders en mi caso el 5 en ShaderList
shaderList[5].useShader();
uniformModel = shaderList[5].getModelLocation();
uniformProjection = shaderList[5].getProjectLocation();
//
//Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geométricas
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
//
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA y se envían al shader
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
meshColorList[0] -> RenderMeshColor();
*/
```

Ejecución



2.- Generar el dibujo de la casa de la clase, pero en lugar de instanciar triángulos y cuadrados será instanciando pirámides y cubos, para esto se requiere crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp.

```
//Vertex Shader
static const char* vShader = "shaders/shader.vert";
static const char* fShader = "shaders/shader.frag";
static const char* vSolid = "shaders/shader.vert";
// Nuevos
static const char* fRed = "shaders/rojo.frag";
static const char* vRed = "shaders/rojo.vert";
static const char* fGreen = "shaders/verde.frag";
static const char* vGreen = "shaders/verde.vert";
static const char* fBlue = "shaders/azul.frag";
static const char* vBlue = "shaders/azul.vert";
static const char* fBrown = "shaders/cafe.frag";
static const char* vBrown = "shaders/cafe.vert";
static const char* fDGreen = "shaders/verde_fuerte.frag";
static const char* vDGreen = "shaders/verde_fuerte.vert";
static const char* vShaderColor = "shaders/shadercolor.vert";
static const char* fShaderColor = "shaders/shadercolor.frag";
```

Lo que hacemos aquí es agregar todos los shader .vert y .frag

Los .vert

```
1  #version 330
2  layout (location =0) in vec3 pos;
3  in vec3 color;
4  out vec4 vColor;
5  uniform mat4 model;
6  uniform mat4 projection;
7  void main()
8  {
9      gl_Position = projection * model * vec4(pos,1.0f);
10     vColor = vec4(0.0f, 0.0f, 1.0f, 1.0f); // Azul
11 }
```

Todos tienen la misma estructura, lo único que cambiar es el color que va a tener.

Para los .frag

```
#version 330
in vec4 vColor;
out vec4 color;
void main()
{
    color = vColor;
}
```

Todos mantienen la misma estructura.

Y por cada color se creo un archivo .frag y .vert

```
Shader* shader1 = new Shader(); //shader para usar indices: objetos: cubo y pirámide
/*shader1->CreateFromFiles(vShader, fShader);
shaderList.push_back(*shader1);*/
Shader* sRed = new Shader(); sRed->CreateFromFiles(vRed, fRed); shaderList.push_back(*sRed);
Shader* sGreen = new Shader(); sGreen->CreateFromFiles(vGreen, fGreen); shaderList.push_back(*sGreen);
Shader* sBlue = new Shader(); sBlue->CreateFromFiles(vBlue, fBlue); shaderList.push_back(*sBlue);
Shader* sBrown = new Shader(); sBrown->CreateFromFiles(vBrown, fBrown); shaderList.push_back(*sBrown);
Shader* sDGrn = new Shader(); sDGrn->CreateFromFiles(vDGreen, fDGreen); shaderList.push_back(*sDGrn);
// [0]=rojo, [1]=verde, [2]=azul, [3]=café, [4]=verde oscuro

Shader* shader2 = new Shader(); //shader para usar color como parte del VAO: letras
shader2->CreateFromFiles(vShaderColor, fShaderColor);
shaderList.push_back(*shader2);
}
```

Aquí lo que hacemos es instanciar a los shaders con sus respectivos archivos y después los agregamos a la lista.

Posteriormente en el main mandamos a llamar a nuestras funciones para que en el while empezamos a formar nuestras figuras, se puede observar la creación de dos variables z que son dos profundidades: base (-4.0) y frontal (-3.9) para que ventanas/puerta queden por delante del cuerpo.

```
/*Creamos estas variables para el control de z y son constantes por que las vamos a estar repitiendo dependiendo de la figura*/
const float Z_BODY = -4.0f; // capa base
const float Z_FRONT = -3.9f; // un poquito más cerca (ventanas/puerta)

glm::mat4 model;

// CUBO ROJO
shaderList[0].useShader(); // rojo
uniformModel = shaderList[0].getModelLocation();
uniformProjection = shaderList[0].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -0.4f, Z_BODY));
model = glm::scale(model, glm::vec3(0.85f, 1.10f, 1.0f)); // ancho, alto, espesor Z
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1]->RenderMesh(); // CUBO

// PIRÁMIDE AZUL
shaderList[2].useShader(); // azul
uniformModel = shaderList[2].getModelLocation();
uniformProjection = shaderList[2].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
```

```

uniformProjection = shaderList[2].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.55f, Z_BODY));
model = glm::scale(model, glm::vec3(1.0f, 0.80f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0] -> RenderMesh(); // PIRAMIDE

shaderList[1].useShader(); // verde
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

// izquierda
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.20f, -0.10f, Z_FRONT));
model = glm::scale(model, glm::vec3(0.30f, 0.30f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1] -> RenderMesh(); // CUBO

// derecha
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.20f, -0.10f, Z_FRONT));
model = glm::scale(model, glm::vec3(0.30f, 0.30f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1] -> RenderMesh(); // CUBO

// CUBO VERDE

```

```

// CUBO VERDE
shaderList[1].useShader(); // verde
uniformModel = shaderList[1].getModelLocation();
uniformProjection = shaderList[1].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, -0.75f, Z_FRONT));
model = glm::scale(model, glm::vec3(0.30f, 0.40f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1] -> RenderMesh(); // CUBO

// pirámide verde
shaderList[4].useShader(); // verde fuerte
uniformModel = shaderList[4].getModelLocation();
uniformProjection = shaderList[4].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.75f, -0.45f, Z_BODY));
model = glm::scale(model, glm::vec3(0.40f, 0.60f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0] -> RenderMesh();

// cubo café
shaderList[3].useShader(); // café
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

```

```

// cubo café
shaderList[3].useShader(); // café
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(-0.75f, -0.85f, Z_BODY));
model = glm::scale(model, glm::vec3(0.20f, 0.20f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1] -> RenderMesh();

shaderList[4].useShader(); // verde fuerte
uniformModel = shaderList[4].getModelLocation();
uniformProjection = shaderList[4].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.75f, -0.45f, Z_BODY));
model = glm::scale(model, glm::vec3(0.45f, 0.60f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[0] -> RenderMesh();

// Tronco
shaderList[3].useShader(); // café
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

```

```

// Tronco
shaderList[3].useShader(); // café
uniformModel = shaderList[3].getModelLocation();
uniformProjection = shaderList[3].getProjectLocation();
glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.75f, -0.85f, Z_BODY));
model = glm::scale(model, glm::vec3(0.20f, 0.20f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshList[1] -> RenderMesh();

```

Entonces lo que hacemos es utilizar el color que tiene nuestro shader que es el que agregamos a la lista ShaderList, de esta manera solo agregamos el índice en el que se encuentra (tomando el orden en que se fueron agregando) y la figura, si meshList[] tiene el índice 1 es un cubo si es 0 entonces es una pirámide, sobre las coordenadas trate de guiarme con las de la práctica pasada para que la casa quedara ordenada.

Importante mencionar, que, al no agregarle una rotación, pareciera que la casa es 2d aunque fue creada con cubos y pirámides.

Ejecución



2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

No hubo problemas con la ejecución del código.

Conclusión:

Los ejercicios de esta práctica me resultaron bastante difíciles, ya que la explicación proporcionada no fue suficiente para comprender completamente lo que debía realizar. Al inicio tuve muchas dudas sobre qué se pedía y cómo debía llevarlo a cabo. La creación de archivos adicionales que contenían prácticamente la misma información me pareció ambigua. Aunque en apariencia la práctica pudiera parecer sencilla, en la realidad me resultó bastante complicada de realizar.

Sin embargo, a medida que avancé y revisé con mayor detalle el código, logré identificar la lógica detrás de cada bloque y comprendí la función de los distintos shaders. Con paciencia y prueba fui ajustando la estructura, hasta conseguir que las figuras y las letras se mostraran correctamente con los colores esperados.